Software Requirements Specification

for

Secure IoT Gateway Simulation for Resource Sensing

Version 1.0 approved

Prepared by Namish Gupta

Vellore Institute of Technology, Chennai

17th July, 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

1.1.1 Product Identification

Product Name: Secure IoT Gateway Simulation for Resource Sensing (SIGS-RS)
Release/Revision Number: Version 1.0.0

1.1.2 Product Scope

The **Secure IoT Gateway Simulation for Resource Sensing** is a backend network software engine designed to model, ingest, and process telemetry data from a theoretical community sensor grid.

This Software Requirements Specification (SRS) covers the **backend simulation engine and the middleware security layer** exclusively. The scope of the product detailed in this document includes:

- **Telemetry Ingestion & Simulation:** The core engine responsible for simulating high-volume, concurrent IoT packet streams representing community resource metrics (e.g., water, energy, or air quality data).
- **The Anomalous Packet Inspector:** A primary subsystem of the middleware layer. This document covers the logic and thresholds for the sliding-window algorithm used to evaluate real-time data rates and payload integrity, as well as the protocols for dropping spoofed or malicious packets.
- **The Privacy Anonymizer:** The secondary subsystem of the middleware layer. This document defines the data-masking pipeline that intercepts verified packets and strips personally identifiable information (PII)—specifically geolocation metadata—prior to database persistence.

Out of Scope: This SRS covers only the backend gateway simulation and middleware components. It **does not** include requirements for physical IoT sensor hardware, firmware development, or the frontend graphical user interface (GUI) used for end-user data visualization.

1.2 Document Conventions

1.2.1 Document Conventions

This Software Requirements Specification (SRS) for the Secure IoT Gateway Simulation adheres to the IEEE 830-1998 standard for structural design and organization. To ensure clarity and readability throughout the document, the following typographical conventions are strictly applied:

- **Bold Text:** Utilized to identify primary system modules, subsystems, and critical architectural components (e.g., **Anomalous Packet Inspector**, **Privacy Anonymizer**).
- *Italic Text:* Applied to highlight specific algorithms, data properties, and internal terminology (e.g., *sliding-window algorithm*, *geolocation metadata*, *telemetry payloads*).
- `Monospace Text`: Reserved for code snippets, API endpoints, data formats, and configuration parameters.

1.2.2 Requirement Prioritization

Regarding the prioritization of functional and non-functional requirements, priorities are **not** assumed to be inherited from higher-level parent categories. To ensure granular control during the development and testing of the middleware security layers, every individual detailed requirement statement is explicitly assigned its own distinct priority level (e.g., High, Medium, or Low) based on its criticality to the system's core objectives.

1.3 Intended Audience and Reading Suggestions

1.3.1 Intended Audience and Document Organization

This Software Requirements Specification (SRS) is intended for a technical and managerial audience involved in the development and lifecycle of the Secure IoT Gateway Simulation. The primary readers include **Backend Developers**, **Security Engineers**, **Quality Assurance (QA) Testers**, and **Project Managers**.

The remainder of this document is organized to progress from a high-level architectural overview to granular technical specifications:

- **Section 2: Overall Description**

Provides context on the system environment, user classes, and general constraints of the community sensor grid.

- **Section 3: System Features**

Contains the core functional requirements, detailing the backend network simulation engine, the *Anomalous Packet Inspector* (including the sliding-window algorithm and threshold metrics), and the *Privacy Anonymizer* (detailing the data-masking pipeline for geolocation metadata).

- **Section 4: Non-Functional Requirements**

Outlines performance expectations, security standards, and system reliability metrics.

1.3.2 Suggested Reading Sequence

To make the most efficient use of this document, we recommend the following reading sequences tailored by role:

- **Project Managers:** Read Sections 1 and 2 to grasp the product scope, operating environment, and overarching system goals without getting immersed in granular algorithmic details.
- **Backend Developers & Security Engineers:** Read the document in its entirety. Focus heavily on Section 3 to understand the exact payload validation rules, data rate thresholds, and geolocation stripping mechanics required for the middleware implementation.
- **QA Testers:** Begin with the overviews in Sections 1 and 2 for context, then concentrate on Sections 3 and 4 to design robust test cases around simulated packet ingestion rates, malicious packet-dropping triggers, and database privacy compliance.

1.4 Product Scope

1.4.1 Product Purpose and Description

The **Secure IoT Gateway Simulation for Resource Sensing** is a backend network simulation engine designed to model a community sensor grid and process high-volume telemetry data. Its primary purpose is to serve as a robust testing environment to validate a custom middleware security layer engineered to sanitize, protect, and anonymize incoming IoT traffic before it reaches persistent storage.

1.4.2 Objectives and Benefits

The system is built to achieve two primary technical objectives:

- **Ensure Data Integrity:** By utilizing the **Anomalous Packet Inspector**, the system protects the backend by evaluating incoming data rates and payloads against baseline thresholds via a sliding-window algorithm, automatically dropping spoofed or malicious packets.
- **Enforce Data Privacy:** By routing verified packets through the **Privacy Anonymizer**, the system ensures user safety by stripping personally identifiable geolocation metadata from the sensing logs prior to database insertion.

1.4.3 Alignment with Business Strategies

This software directly supports the corporate strategic goal of pioneering secure, privacy-first smart-community infrastructure. By utilizing a software-based simulation engine rather than deploying phys-

ical hardware for early-stage testing, the organization significantly reduces R&D overhead, accelerates iterative testing, and ensures that the eventual physical deployment complies with stringent data privacy regulations.

*Note: For a comprehensive breakdown of the long-term strategic roadmap, market positioning, and financial objectives, please refer to the separate **Secure IoT Gateway Vision and Scope Document**.*

1.5 References

1.5.1 Referenced Documents

The following documents and standards are referenced throughout this Software Requirements Specification. They provide essential context, strategic direction, and compliance guidelines for the Secure IoT Gateway Simulation:

- **Secure IoT Gateway Vision and Scope Document**
  - **Author:** Enterprise Strategy Team
  - **Version:** 1.2
  - **Date:** March 15, 2026
  - **Location:** Internal Corporate SharePoint Repository ([`https://intranet.corp/docs/vision-scope-v1.2.pdf`])
  - **Description:** Outlines the long-term business strategy, financial objectives, and market positioning for the broader smart-community infrastructure project.
- **IEEE Recommended Practice for Software Requirements Specifications**
  - **Author:** IEEE Computer Society
  - **Version:** IEEE Std 830-1998
  - **Date:** October 20, 1998
  - **Location:** IEEE Xplore Digital Library ([`https://ieeexplore.ieee.org/document/720574`])
  - **Description:** Defines the structural and typographical standards adhered to in the creation of this SRS document.
- **Data Anonymization and Privacy Compliance Guidelines**
  - **Author:** Corporate Security and Legal Department
  - **Version:** 3.0
  - **Date:** January 10, 2026
  - **Location:** Central Engineering Wiki ([`https://wiki.corp/security/privacy-guidelines-v3`])
  - **Description:** Dictates the specific regulatory requirements and internal protocols used

by the **Privacy Anonymizer** pipeline to successfully strip personally identifiable geolocation metadata.

- **Community Sensor Grid API Specification**
  - **Author:** IoT Integration Team
  - **Version:** 1.0.4
  - **Date:** June 5, 2026
  - **Location:** Internal GitHub Repository ([`https://github.corp/iot-team/sensor-grid-api/docs`])
  - **Description:** Details the payload structures and transmission rates for the external virtual data generator that interfaces with the simulation engine, providing the baseline metrics required by the **Anomalous Packet Inspector**.

2. Overall Description

2.1 Product Perspective

2.1.1 Product Context and Origin

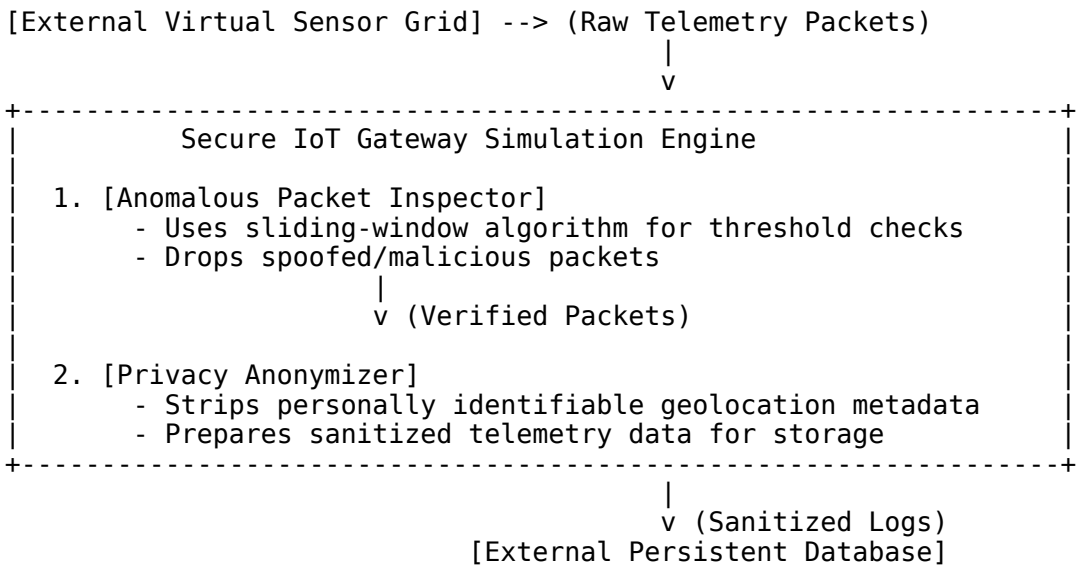
The **Secure IoT Gateway Simulation for Resource Sensing** is a new, self-contained software product acting as the foundational R&D testing environment for a broader, upcoming physical smart-community grid. Rather than replacing an existing system, this simulation engine acts as a precursor component to the larger enterprise physical gateway system. It allows developers to rigorously test high-volume telemetry processing and the custom middleware security layer in a risk-free, simulated environment before committing to expensive hardware deployments.

2.1.2 System Interfaces and Interconnections

Within the overarching enterprise architecture, this software serves as the intermediary routing and filtering layer. It interfaces with an external virtual data generator (simulating the physical IoT community sensors) via standard MQTT/HTTP protocols, and outputs sanitized telemetry logs to an external persistent database. The internal architecture dictates that all ingested traffic must sequentially pass through the middleware's two primary security subsystems—the **Anomalous Packet Inspector** and the **Privacy Anonymizer**—before reaching the database interface.

2.1.3 Architectural Diagram

Below is a simplified structural representation of the subsystem interconnections, external interfaces, and the directional flow of telemetry data:



2.2 Product Functions

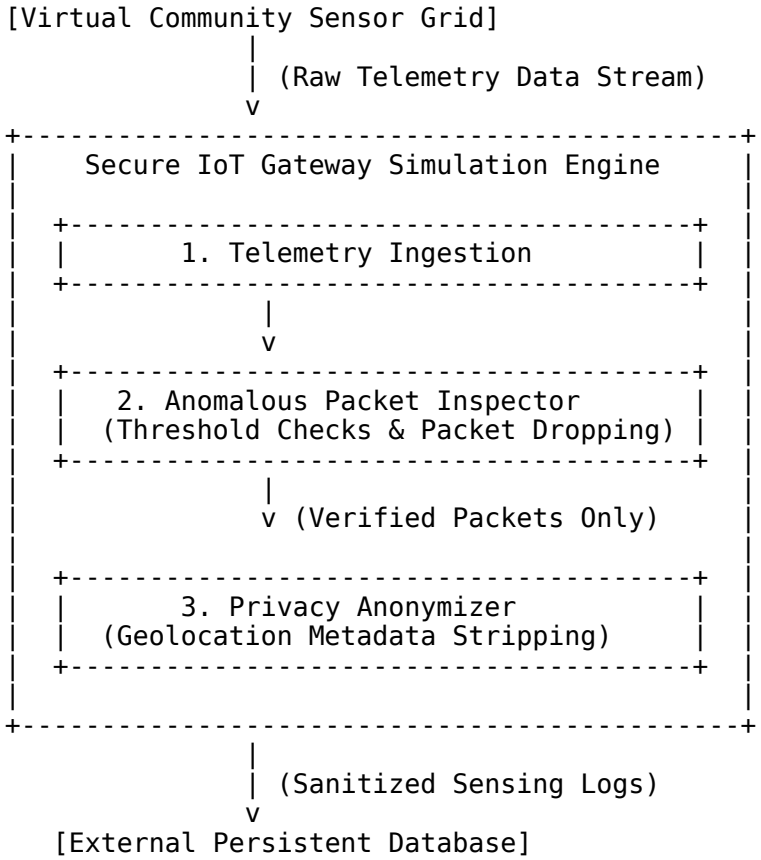
2.2.1 Major System Features and Functions

The Secure IoT Gateway Simulation for Resource Sensing operates primarily as a backend processing and routing engine. The major functions the system performs can be summarized into the following high-level capabilities:

- **Telemetry Ingestion:** Receives and models high-volume, concurrent simulated packet streams representing community resource metrics (e.g., water, energy, air quality).
- **Real-Time Threat Detection:** Utilizes the **Anomalous Packet Inspector** to continuously monitor incoming data rates and evaluate payload integrity against predefined baseline thresholds.
- **Automated Threat Mitigation:** Automatically drops spoofed or malicious packets that exceed the anomaly thresholds calculated by the internal *sliding-window algorithm*.
- **Data Sanitization:** Routes verified packets through the **Privacy Anonymizer** pipeline to detect and strip personally identifiable information (PII), specifically targeting sensitive *geolocation metadata*.
- **Secure Data Persistence:** Forwards the fully sanitized and verified community sensing logs to the external persistent database for storage and future analysis.

2.2.2 High-Level Data Flow Diagram

The following diagram illustrates the major functional groups of the system and how telemetry data flows through the simulation engine and middleware layers:



2.3 User Classes and Characteristics

2.3.1 User Classes and Characteristics

Because the Secure IoT Gateway Simulation operates exclusively as a backend processing engine and middleware layer, its users are primarily technical personnel involved in the development, security configuration, and testing of the system. There are no traditional layperson end-users for this software. The user classes are categorized by their technical roles, privilege levels, and the specific subsystems they interact with.

The user classes are distinguished below, with **Security Engineers** and **Backend Developers** designated as the most important primary users to satisfy, as their requirements dictate the core success of the middleware.

2.3.1.1 Primary User Classes

- **Security Engineers:**
  - **Characteristics:** Highly technical users with deep expertise in network security, threat modeling, and data privacy regulations.
  - **System Interaction:** This is the most critical user class. They are responsible for defin-

ing the baseline thresholds and configuration parameters for the **Anomalous Packet Inspector**. They dictate the rules for the *sliding-window algorithm* and define the specific *geolocation metadata* parameters that the **Privacy Anonymizer** must strip.

- **Privilege Level:** High. They require administrative access to modify security configurations and audit the sanitized telemetry logs.

- **Backend Developers:**
  - **Characteristics:** Experienced software engineers responsible for system architecture, API integration, and data pipeline management.
  - **System Interaction:** They interface directly with the simulation engine to integrate external virtual sensor grids and connect the output pipeline to the persistent database. They require clear, programmatic access to system logs to debug the routing of *telemetry payloads* through the middleware layers.
  - **Privilege Level:** High. They require full access to the codebase, simulation environment, and database schemas.

2.3.1.2 Secondary User Classes

- **Quality Assurance (QA) Testers:**
  - **Characteristics:** Technical users focused on system validation, stress testing, and edge-case discovery.
  - **System Interaction:** They utilize the simulation engine to inject high volumes of standard traffic mixed with spoofed or malicious packets. Their primary goal is to validate that the *sliding-window algorithm* drops malicious data at scale and that no unmasked *geolocation metadata* leaks into the database.
  - **Privilege Level:** Medium. They need execution privileges for the simulation engine and read-only access to the database to verify outputs.

- **DevOps / System Administrators:**
  - **Characteristics:** Infrastructure specialists focused on deployment, uptime, and system performance.
  - **System Interaction:** They are responsible for deploying the simulation engine within the corporate testing environment, allocating computational resources for high-volume ingestion, and maintaining the connections to external databases.
  - **Privilege Level:** Medium to High. They control the deployment environment but do not typically interact with the internal middleware algorithms.

2.4 Operating Environment

2.4.1 Operating Environment

The Secure IoT Gateway Simulation is a highly concurrent backend service designed to process high-volume telemetry traffic. It is engineered to operate within modern, scalable server environments and relies on containerization for consistent deployment.

2.4.1.1 Hardware Platform

The software does not run on physical IoT edge devices; instead, it requires robust backend infrastructure to simulate the heavy load of a community sensor grid.

- **Architecture:** x86_64 or ARM64 multi-core server architectures.
- **Minimum Simulation Specs:** A minimum of 16 vCPUs and 32 GB of RAM is recommended to ensure the **Anomalous Packet Inspector** can compute the *sliding-window algorithm* across thousands of concurrent connections without bottlenecking the ingestion pipeline.

2.4.1.2 Operating Systems

The system is built as a platform-agnostic, containerized application, but native deployment is officially supported and tested on Linux environments:

- **Primary OS:** Ubuntu 22.04 LTS (or newer).
- **Secondary OS:** CentOS Stream 9 or Alpine Linux (for lightweight container builds).

2.4.1.3 Software Ecosystem and Coexistence

The simulation engine acts as an intermediary node and must peacefully coexist and interface with several external components and third-party software layers:

- **Container Orchestration:** The engine runs within Docker containers and is designed to be orchestrated by Kubernetes (K8s) for horizontal scaling during stress-testing scenarios.
- **Message Brokers:** The ingestion layer must interface smoothly with standard IoT message brokers, primarily MQTT (e.g., Eclipse Mosquitto or EMQX) and HTTP REST APIs, to receive data from the external virtual sensor grid.
- **Persistent Storage Engine:** Once telemetry passes through the **Privacy Anonymizer**, the output pipeline connects directly to time-series or relational databases, such as TimescaleDB, InfluxDB, or PostgreSQL, to store the sanitized logs.
- **Virtual Data Generators:** The system coexists alongside Python or Go-based load-generation scripts that act as the simulated community sensor nodes, continuously pushing both legitimate and malicious *telemetry payloads* into the network.

2.5 Design and Implementation Constraints

2.5.1 Design and Implementation Constraints

The development and architecture of the Secure IoT Gateway Simulation are bound by several strict technical, regulatory, and operational constraints:

- **Regulatory and Compliance Mandates:** The **Privacy Anonymizer** pipeline must strictly adhere to the internal *Data Anonymization and Privacy Compliance Guidelines* (Version 3.0). Because the system simulates real-world community sensing logs, the stripping of personally identifiable *geolocation metadata* must be absolute to comply with enterprise data protection standards modeling real-world regulations (e.g., GDPR, CCPA).
- **Algorithmic Performance Limits:** The **Anomalous Packet Inspector** utilizes a stateful *sliding-window algorithm* to track real-time telemetry rates. This imposes severe memory and timing constraints. Developers must ensure the in-memory state tracking for thousands of concurrent virtual sensors does not exceed the maximum 32 GB RAM allocation or delay the ingestion pipeline by more than 15 milliseconds per packet.
- **Protocol Restrictions:** Inbound communication interfaces are strictly limited to MQTT and HTTP/HTTPS REST protocols to accurately mirror the eventual physical edge-device capabilities. Proprietary or non-standard IoT protocols cannot be used for the external virtual data generator interfaces.
- **Architectural Mandates:** The software must be fully containerized (Docker) and orchestration-ready (Kubernetes). Developers are constrained from using any OS-specific or hardware-dependent libraries that would break the platform-agnostic, microservices-based deployment model.
- **Database Interface Standards:** The output pipeline must be written to interface exclusively with the organization's pre-approved standard for persistent storage (PostgreSQL/TimescaleDB). The sanitized data schemas must perfectly match the existing upstream analytical database tables.

2.6 User Documentation

Because the Secure IoT Gateway Simulation (SIGS-RS) operates exclusively as a backend processing engine and middleware layer, traditional layperson end-user manuals will not be provided. Instead, the user documentation is tailored for technical personnel (Security Engineers, Backend Developers, QA Testers, and DevOps) and focuses on deployment, API integration, and security configuration.

The following documentation components will be delivered along with the software:

- **Middleware Configuration Guide:** A technical manual detailing the parameter tuning for

the security layers. This includes defining baseline data-rate thresholds for the **Anomalous Packet Inspector** and mapping the specific JSON paths for *geolocation metadata* that the **Privacy Anonymizer** must strip.

- **System Integration & API Specification:** Developer-focused documentation outlining the expected MQTT/HTTP input schemas from the virtual sensor grid and the schemas for the sanitized data output to the persistent database.
- **Deployment and Orchestration Readme:** A step-by-step infrastructure guide covering Docker container build processes, environment variable configurations, and Kubernetes (K8s) deployment manifests.

Delivery Formats and Standards:

- **Markdown (.md):** All deployment instructions and core configuration guides will be written in standard Markdown and maintained directly within the project's Git version control repository.
- **OpenAPI (Swagger) 3.0:** The API specifications for telemetry ingestion will be generated and hosted using the OpenAPI standard, allowing developers to interactively test payload endpoints.
- **Enterprise Wiki (Confluence):** High-level architectural overviews, threat-modeling diagrams, and release notes will be published to the internal corporate wiki using standard engineering templates.

2.7 Assumptions and Dependencies

2.7.1 Assumptions

The successful deployment, execution, and validation of the Secure IoT Gateway Simulation (SIGS-RS) rely on the following assumed factors:

- **Data Generator Reliability:** It is assumed that the external virtual sensor grid (the load-generation scripts) is capable of sustaining high-volume, concurrent payload generation without localized crashing or bottlenecking. If the generator fails, the simulation engine's stress-testing capabilities cannot be accurately validated.
- **Payload Consistency:** The raw telemetry payloads transmitted via MQTT/HTTP will adhere to the baseline JSON structural schemas defined in the *Community Sensor Grid API Specification*.
- **Database I/O Capacity:** The external persistent database (e.g., PostgreSQL or TimescaleDB) is assumed to be provisioned with sufficient compute and write-speed capacity to ingest the sanitized logs continuously. Insufficient database I/O will cause backpressure on the **Privacy Anonymizer** pipeline.

2.7.2 Dependencies

The project has strict dependencies on the following external systems, software components, and organizational guidelines:

- **Infrastructure Availability:** The deployment of the simulation engine is entirely dependent on an available, pre-configured Kubernetes (K8s) cluster and a Docker container registry within the enterprise testing environment.
- **Message Broker Uptime:** Telemetry ingestion depends on the stability of a third-party MQTT broker (such as Eclipse Mosquitto or EMQX) to handle the initial routing of packets from the virtual grid to the gateway.
- **Compliance Framework:** The specific fields stripped by the **Privacy Anonymizer** are strictly dependent on the *Data Anonymization and Privacy Compliance Guidelines (Version 3.0).* Any subsequent updates to enterprise privacy policies will require immediate refactoring of the anonymization pipeline.

3. External Interface Requirements

3.1 User Interfaces

3.1.1 User Documentation

Because the Secure IoT Gateway Simulation is a backend R&D tool intended for technical personnel (Security Engineers, Backend Developers, and QA Testers), the delivered documentation will focus on system configuration, API integration, and deployment rather than traditional end-user manuals.

The following documentation components will be delivered alongside the software:

- **Middleware Configuration Guide:** A comprehensive technical manual detailing the parameter tuning for the security layers. It will include instructions on defining baseline thresholds for the **Anomalous Packet Inspector**'s *sliding-window algorithm* and mapping the specific rules for the **Privacy Anonymizer** to successfully identify and strip *geolocation metadata*.
- **System Integration & API Specification:** Developer-focused documentation outlining the expected MQTT/HTTP input schemas from the virtual sensor grid and the structural schemas for the sanitized data output to the persistent database.
- **Deployment and Orchestration Readme:** A step-by-step guide for DevOps and system administrators covering the Docker container build process, environment variable configuration, and Kubernetes (K8s) deployment manifests for scaling the simulation engine.

3.1.2 Delivery Formats and Standards

To ensure seamless integration with the engineering team's existing workflows, the documentation will adhere to the following delivery formats:

- **Repository-Level Documentation:** All deployment instructions and core configuration guides will be written in standard Markdown (.md) format and stored directly within the project's version control repository.
- **API Documentation:** The System Integration & API Specification will be generated and hosted using the OpenAPI (Swagger) 3.0 standard, allowing developers to interactively test payload ingestion endpoints.
- **Internal Wiki Integration:** High-level architectural overviews and release notes will be published to the corporate Confluence/Wiki, conforming to the enterprise engineering documentation templates.

3.2 Hardware Interfaces

Because the Secure IoT Gateway Simulation (SIGS-RS) is exclusively a backend network software engine, it does not interface directly with physical IoT edge devices, environmental sensors, or localized hardware controllers (e.g., GPIO, I2C, or serial interfaces). All community sensing hardware is strictly theoretical and modeled by external load-generation software.

The application interfaces with hardware only abstractly through the host operating system and container runtime (Docker). The logical hardware dependencies include:

- **Network Interface Controllers (NICs):** The system relies on the host server's standard enterprise network adapters to receive high-throughput TCP/IP traffic (MQTT and HTTP payloads) from the virtual sensor grid and to transmit sanitized logs to the external database.
- **Compute and Memory Allocation:** The **Anomalous Packet Inspector** relies heavily on the host hardware's CPU and RAM to maintain the stateful tracking of the *sliding-window algorithm*. The software requires the hardware to provide standard x86_64 or ARM64 multi-core processing, with a minimum allocation of 16 vCPUs and 32 GB of RAM to prevent hardware-induced latency or unintended packet dropping.

3.3 Software Interfaces

The Secure IoT Gateway Simulation (SIGS-RS) operates as a middleware routing engine and must interface seamlessly with several external software components to ingest, process, and persist telemetry data.

- **MQTT Message Broker:**

- **Component:** Eclipse Mosquitto or EMQX (Version 2.0 or higher).
- **Purpose:** Acts as the primary ingestion gateway. The simulation engine subscribes to predefined MQTT topics to receive the high-frequency *telemetry payloads* generated by the external virtual sensor grid.
- **Data Flow (Inbound):** The system receives raw, JSON-formatted messages containing simulated resource metrics (e.g., timestamp, sensor ID, resource type, payload values, and raw *geolocation metadata*).

- **Persistent Storage Engine (Database):**
  - **Component:** PostgreSQL 15+ or TimescaleDB.
  - **Purpose:** Serves as the long-term storage destination for data that has successfully passed through both the **Anomalous Packet Inspector** and the **Privacy Anonymizer**.
  - **Data Flow (Outbound):** The simulation engine utilizes standard database connection pooling to INSERT sanitized, verified telemetry logs. The outbound data schema strictly matches the upstream analytical tables and is completely stripped of personally identifiable information.
- **Container Orchestration (Kubernetes API):**
  - **Component:** Kubernetes (K8s) Control Plane.
  - **Purpose:** The engine exposes a lightweight HTTP REST interface (e.g., `/health` and `/metrics`) to allow the orchestration software to monitor system health, measure ingestion latency, and trigger horizontal pod autoscaling during peak stress-testing scenarios.

3.4 Communications Interfaces

The Secure IoT Gateway Simulation (SIGS-RS) requires robust, low-latency, and highly secure communication channels to handle continuous telemetry ingestion and data persistence. All data in transit must comply with enterprise encryption standards.

- **Ingestion Protocols (MQTT):**
  - The primary communication method between the external virtual sensor grid and the simulation engine is MQTT (Message Queuing Telemetry Transport) over TCP/IP (Port 8883).
  - All MQTT traffic must be encrypted using TLS 1.2 or higher (TLS 1.3 preferred) using X.509 client certificates to authenticate the virtual grid connections before they reach the **Anomalous Packet Inspector**.
- **RESTful API Communications (HTTP/HTTPS):**
  - The system exposes management, health-check, and configuration endpoints via HTTP/HTTPS (Port 443).
  - Orchestration tools (like Kubernetes) poll the `/health` and `/metrics` endpoints via

standard HTTP GET requests.

- Security engineers updating the **Privacy Anonymizer** configurations will interface via HTTPS REST endpoints authenticated using OAuth 2.0 and JSON Web Tokens (JWT).

- **Database Communications:**
  - The simulation engine communicates with the external persistent database (e.g., PostgreSQL or TimescaleDB) utilizing native TCP-based connection pooling (typically Port 5432).
  - This internal network traffic must be routed through a secure virtual private cloud (VPC) subnet, utilizing strict SSL mode (`sslmode=verify-full`) to prevent internal packet sniffing of the sanitized logs.

4. System Features

This section details the core functional requirements of the Secure IoT Gateway Simulation for Resource Sensing (SIGS-RS). To best reflect the system's architectural pipeline and the sequential flow of telemetry data, these functional requirements are logically organized by middleware subsystem.

The requirements are divided into the system's three primary processing layers:

- **4.1 Telemetry Ingestion:** The baseline capabilities for receiving, parsing, and queuing high-volume concurrent payload streams from the virtual sensor grid.
- **4.2 Anomalous Packet Inspector:** The sliding-window algorithms, baseline threshold metrics, and automated packet-dropping protocols used to identify and mitigate malicious traffic.
- **4.3 Privacy Anonymizer:** The compliance and data-masking pipeline responsible for identifying and stripping personally identifiable geolocation metadata prior to database persistence.

4.1 Telemetry Ingestion Pipeline

4.1.1 Description and Priority

This feature encompasses the core capability to securely receive, parse, and queue high-volume concurrent telemetry streams (via MQTT and HTTP) generated by the external virtual sensor grid. **Priority: High.** It is the foundational entry point for all data entering the simulation engine; without it, downstream middleware testing cannot occur.

4.1.2 Stimulus/Response Sequences

Stimulus: The virtual sensor grid initiates a secure MQTT connection and transmits a batch of JSON telemetry payloads.

Response: The system authenticates the TLS certificate, accepts the payload, and routes it to the internal ingestion queue.
Stimulus: An external client attempts to send malformed JSON or exceeds the maximum payload size limit.
Response: The system immediately drops the connection, discards the packet, and logs an ingestion error code without passing data to the middleware.

4.1.3 Functional Requirements

REQ-TI-1: The system shall accept secure MQTT connections exclusively on Port 8883 using TLS 1.2 or higher.

REQ-TI-2: The system shall successfully parse incoming telemetry payloads formatted strictly as JSON according to the Community Sensor Grid API schema.

REQ-TI-3: The system shall reject and drop any individual telemetry packet that exceeds a maximum payload size of 512 KB to prevent memory exhaustion.

REQ-TI-4: The system shall enqueue verified, successfully parsed packets into an in-memory buffer for sequential processing by the Anomalous Packet Inspector.

4.2 Anomalous Packet Inspector

4.2.1 Description and Priority

The Anomalous Packet Inspector is the primary security middleware subsystem. It utilizes an in-memory sliding-window algorithm to evaluate the frequency and integrity of incoming telemetry payloads. Its purpose is to detect and mitigate spoofing attacks, Denial of Service (DoS) attempts, and corrupted sensor data before it penetrates further into the network. **Priority: High.**

4.2.2 Stimulus/Response Sequences

Stimulus: The ingestion pipeline forwards a queued telemetry packet to the Inspector.
Response: The algorithm calculates the current packet rate for the originating sensor ID. If the rate and payload values fall within baseline thresholds, the packet is flagged as ’Verified’ and forwarded to the Privacy Anonymizer.
Stimulus: A sensor exceeds the defined transmission rate (e.g., a burst of 100 packets in one second) or submits statistically impossible resource metrics.
Response: The Inspector immediately drops the offending packet(s) from memory and records a violation in the internal security log.

4.2.3 Functional Requirements

REQ-API-1: The system shall maintain an in-memory state tracking the timestamp of all packets received from each unique sensor ID over a rolling 10-second sliding window.

REQ-API-2: The system shall drop any incoming packet if the sliding window indicates the originating sensor ID has exceeded a transmission rate of 50 packets per 10 seconds.

REQ-API-3: The system shall evaluate the internal payload metrics and drop packets containing impossible environmental state variables (e.g., negative power consumption, or air quality indices exceeding standard bounds).

REQ-API-4: The system shall compute the sliding-window validation and payload evaluation for each packet in under 15 milliseconds to prevent ingestion pipeline bottlenecks.

REQ-API-5: Upon dropping a packet, the system shall silently discard the data without sending a rejection acknowledgment to the external client, mitigating network reflection attacks.

4.3 Privacy Anonymizer

4.3.1 Description and Priority

The Privacy Anonymizer is the secondary subsystem of the middleware layer. It functions as a strict compliance and data-masking pipeline that intercepts verified telemetry packets to detect and strip personally identifiable information (PII)—specifically sensitive geolocation metadata—before the logs are persisted to the database. **Priority: High.** Failure to strip this data violates enterprise privacy compliance guidelines.

4.3.2 Stimulus/Response Sequences

Stimulus: The Anomalous Packet Inspector forwards a 'Verified' telemetry packet to the Anonymizer pipeline.
Response: The system parses the JSON payload, identifies restricted keys (e.g., `latitude`, `longitude`, `mac_address`), removes them, and forwards the sanitized payload to the database connection pool.
Stimulus: The system encounters a payload structured in an unrecognized format where targeted PII keys cannot be safely isolated.
Response: The system defaults to a fail-secure state, dropping the entire packet to prevent accidental PII leakage into persistent storage, and logging a schema compliance error.

4.3.3 Functional Requirements

REQ-PA-1: The system shall intercept all verified packets and apply a JSON masking function to eliminate `latitude`, `longitude`, and device-specific identifier fields.

REQ-PA-2: The system shall execute the data-masking operations in strict adherence to the rules outlined in the *Data Anonymization and Privacy Compliance Guidelines (Version 3.0)*.

REQ-PA-3: The system shall format the successfully sanitized telemetry data into SQL `INSERT` queries compatible with the persistent PostgreSQL/TimescaleDB analytical schemas.

REQ-PA-4: The system shall implement a fail-secure mechanism that automatically drops any packet if the JSON masking function throws a parsing exception, ensuring no raw data bypasses the filter.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The performance of the Secure IoT Gateway Simulation (SIGS-RS) is critical. It must accurately model the high-stress environment of a physical community sensor grid while performing real-time security and privacy analysis. The system is designed to handle high concurrency with strict latency boundaries to prevent ingestion backpressure.

REQ-PF-1: **Throughput Capacity:** The simulation engine shall successfully ingest and process a sustained load of 50,000 telemetry packets per second from up to 10,000 concurrently connected virtual sensors.

REQ-PF-2: **Processing Latency:** The combined middleware pipeline (Anomalous Packet Inspector and Privacy Anonymizer) shall complete the validation, metadata stripping, and database queuing for any individual packet in under 15 milliseconds.

REQ-PF-3: **Memory Efficiency:** The stateful tracking required by the sliding-window algorithm shall be memory-optimized to operate stably utilizing no more than 24 GB of active RAM, ensuring safe overhead within the 32 GB minimum system environment to prevent Out-Of-Memory (OOM) failures.

REQ-PF-4: **Horizontal Scalability:** The containerized system shall be capable of stateless scaling, triggering Kubernetes horizontal pod autoscaling (HPA) events when sustained CPU utilization exceeds 75% for more than 2 minutes.

5.2 Safety Requirements

Because the Secure IoT Gateway Simulation (SIGS-RS) operates strictly as a backend software simulation environment, there are no direct cyber-physical safety risks (e.g., bodily harm, physical equipment damage, or actuator malfunction) associated with its operation. ”Safety” in this context pertains strictly to the protection of the enterprise infrastructure and the integrity of organizational data from runaway testing loads.

REQ-SF-1: **Environment Isolation:** The simulation engine shall only accept database connection strings targeting designated staging or testing clusters. The system must prevent the accidental injection of simulated telemetry data into live production environments to ensure data integrity.

REQ-SF-2: **Infrastructure Protection (Circuit Breaker):** To prevent the simulation from inadvertently launching a Denial of Service (DoS) attack against the internal corporate network, the system shall implement a circuit breaker that immediately halts database ingestion and drops all incoming MQTT connections if outbound write latency exceeds 2000 milliseconds.

REQ-SF-3: **Fail-Closed Architecture:** In the event of a catastrophic process failure (e.g., a crash within the Privacy Anonymizer container), the system must default to a fail-closed state where all routing ceases entirely, ensuring no unverified or unanonymized raw payloads can bypass the middleware.

5.3 Security Requirements

The security posture of the Secure IoT Gateway Simulation (SIGS-RS) is a core structural component. It is designed to enforce strict data privacy, prevent unauthorized configuration changes, and protect simulated telemetry data in transit across the enterprise network.

REQ-SE-1: **Encryption in Transit:** All external telemetry ingestion via MQTT must be encrypted using TLS 1.2 or higher, requiring valid X.509 client certificates for virtual sensor authentication prior to payload processing.

REQ-SE-2: **API Access Control:** Access to the system’s HTTP/HTTPS management and configuration endpoints shall be strictly authenticated using OAuth 2.0 and JSON Web Tokens (JWT). Unauthenticated requests must be immediately rejected with an HTTP 401 Unauthorized status.

REQ-SE-3: **Database Connection Security:** The system shall communicate with the persistent storage database exclusively over a secure Virtual Private Cloud (VPC) subnet utilizing strict SSL mode (`sslmode=verify-full`).

REQ-SE-4: **Audit Logging:** The system shall maintain an immutable, time-stamped security audit log recording all unauthorized API access attempts, dropped packet events generated by

the Anomalous Packet Inspector, and any configuration changes made by Security Engineers.

5.4 Software Quality Attributes

The Secure IoT Gateway Simulation (SIGS-RS) must satisfy several critical quality attributes to ensure it remains a robust, easily manageable testing environment.

REQ-QA-1: **Reliability:** The simulation engine shall sustain 99.9% uptime during scheduled stress-testing windows. In the event of an isolated service failure, the containerized architecture must utilize Kubernetes self-healing capabilities to restart and restore middleware routing operations within 30 seconds.

REQ-QA-2: **Maintainability:** To facilitate rapid iteration and easy explanation during project reviews, the source code shall adhere to a strictly minimal design standard. The codebase must avoid verbose inline comments and utilize highly concise, simple variable naming conventions, ensuring the underlying routing logic remains accessible, straightforward, and fast to type.

REQ-QA-3: **Portability:** The system shall be entirely platform-agnostic at the operating system level, encapsulated completely within standard Docker containers to guarantee identical execution across local development environments and enterprise testing clusters.

REQ-QA-4: **Testability:** The core middleware subsystems (Anomalous Packet Inspector and Privacy Anonymizer) shall be architected as decoupled modules, allowing for independent unit testing of the threshold calculations and JSON-masking pipelines without requiring a live MQTT broker connection.

5.5 Business Rules

The operation and management of the Secure IoT Gateway Simulation (SIGS-RS) are governed by the following business rules and enterprise policies. These rules dictate role-based access and overarching system behaviors.

BR-1: **Configuration Authority:** Only authenticated personnel formally designated with the 'Security Engineer' role shall have the authority to modify the baseline thresholds of the Anomalous Packet Inspector or alter the specific PII-stripping parameters within the Privacy Anonymizer.

BR-2: **Mandatory Processing Pipeline:** Under no operational circumstances shall telemetry payloads bypass the middleware security layers. All incoming data must be sequentially verified by the Inspector and sanitized by the Anonymizer before database insertion is permitted.

BR-3: **Compliance Lockout:** If the enterprise *Data Anonymization and Privacy Compliance Guidelines* are revised, all simulation testing must be immediately suspended. Testing may only resume after the Privacy Anonymizer pipeline is updated and formally audited by the

compliance team to reflect the new regulations.

BR-4: **Audit Accountability:** All system configuration modifications made via the management APIs must be permanently tied to the individual's enterprise identity in the audit log; anonymous or shared-account administrative changes are strictly prohibited.

6. Other Requirements

6.1 Database and Archival Requirements

The sanitized telemetry logs must be stored in a time-series optimized database architecture (specifically PostgreSQL equipped with the TimescaleDB extension). The data must be partitioned by timestamp (hyper-tables) to ensure that querying high-volume historical simulation data does not degrade the active ingestion performance.

6.2 Reuse Objectives

A core engineering objective of this simulation project is software reuse. The logic, algorithms, and threshold mechanisms developed for the **Anomalous Packet Inspector** and the **Privacy Anonymizer** must be architected as modular, decoupled microservices. This ensures the middleware layers can be directly ported to the physical edge gateway hardware during the next phase of enterprise deployment with minimal refactoring.

Appendix A: Glossary

- **MQTT (Message Queuing Telemetry Transport):** A lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth networks, acting as the primary ingestion protocol for the simulation.
- **PII (Personally Identifiable Information):** Any data that could potentially identify a specific individual. In the context of this system, it primarily refers to sensor *geolocation metadata* (e.g., latitude, longitude, and MAC addresses).
- **SIGS-RS:** Secure IoT Gateway Simulation for Resource Sensing (the system defined in this SRS).
- **Sliding-Window Algorithm:** A computational data structure used by the Anomalous Packet Inspector to maintain a rolling subset of recent telemetry data. It evaluates real-time transmission rates over a fixed timeframe to detect anomalies without retaining long-term state.
- **Telemetry Payload:** The JSON-formatted data packets containing simulated environmental readings and sensor metrics.

Appendix B: Analysis Models

Detailed UML sequence diagrams, entity-relationship diagrams for the TimescaleDB output schemas, and specific state-transition models for the Anomalous Packet Inspector's packet-dropping logic are maintained dynamically in the internal Enterprise Architecture Wiki.

For the high-level structural representation of subsystem interconnections and the directional flow of telemetry data, refer to the **High-Level Data Flow Diagram** presented in **Section 2.2.2** of this document.

Appendix C: To Be Determined List

- **TBD-1:** Finalize the exact JSON path schema definitions for the upcoming version (v1.1) of the external virtual sensor grid.
- **TBD-2:** Determine the absolute maximum connection pool limits for the external persistent PostgreSQL database during peak stress-testing scenarios (50,000+ packets per second).
- **TBD-3:** Define the specific OAuth 2.0 Identity Provider (IdP) endpoints that will be utilized to authenticate Security Engineers accessing the HTTP management APIs.